

Smart PDF Research Assistant

Retrieval-Augmented Document Intelligence

A Simple Technical Guide to an AI-Powered Research and Document Analysis System

Attribute	Detail
Project Category	Full-Stack AI Research & Productivity System
Technology Stack	React, FastAPI, MongoDB, ChromaDB, LangChain, Google Gemini API
Development Context	Independent full-stack AI engineering project

Contents

Abstract.....	3
1 Project Objectives.....	4
2 Theoretical Foundation.....	4
3 Retrieval-Augmented Generation (RAG) Overview.....	4
4 LangChain Framework: Why and How It Is Used.....	4
5 System Architecture.....	5
6 Technology Stack.....	6
7 Document Processing Pipeline.....	6
8 Text Chunking Strategy.....	6
9 Embedding Generation.....	6
10 Vector Database Architecture (ChromaDB).....	6
11 Semantic Search Mechanism.....	6
12 LLM-Based Question Answering.....	7
13 Context Retrieval Workflow.....	7
14 Backend Implementation (FastAPI).....	7
15 Frontend Implementation (React.js).....	7
16 Performance Evaluation.....	7
17 Security and Privacy Considerations.....	8
18 Use Cases.....	8
19 Advantages and Benefits.....	8
20 Limitations.....	8
21 Future Scope.....	8
22 Conclusion.....	9
23 References.....	9

Abstract

The Smart PDF Research Assistant lets people chat with their PDF files instead of reading them page by page. It uses a method called **Retrieval-Augmented Generation (RAG)**, so every answer comes from the actual document, with the page number shown as proof.

It is built with **React**, **FastAPI**, **MongoDB**, and **ChromaDB**, and **LangChain** connects all the AI steps together. **Google Gemini** reads the question and writes the answer.

Keywords: *RAG, LangChain, ChromaDB, Embeddings, Semantic Search, Gemini, FastAPI, React.*

1 Project Objectives

- Find facts inside a PDF instantly, without reading the whole file.
- Let users ask questions in plain language and get clear answers.
- Show the exact page and text used for every answer, so nothing is made up.
- Create quizzes and interview questions from the document automatically.
- Let users search across many PDFs at once with a single question.

2 Theoretical Foundation

This project is built on two ideas: **Natural Language Processing (NLP)** and **Large Language Models (LLMs)**.

- An **LLM** is an AI model that can read and write natural language.
- An **LLM** only knows what it was trained on. It does not know about a PDF the user just uploaded.
- If asked about a private file, an **LLM** may guess and give a wrong answer. This is called a **hallucination**.

To fix this, the system gives the **LLM** the real text from the PDF before asking it to answer.

3 Retrieval-Augmented Generation (RAG) Overview

RAG means the AI is not allowed to answer from memory. It must use the document. It works in three simple steps:

- Retrieve: find the parts of the PDF that match the question.
- Augment: add those parts into the question as "context" for the AI.
- Generate: ask the AI to answer only using that context.

This keeps every answer grounded in the real document instead of a guess.

4 LangChain Framework: Why and How It Is Used

LangChain is the tool that connects every AI step in this project — reading the PDF, splitting the text, creating **embeddings**, searching, and generating the answer — into one simple flow.

4.1 Why LangChain Was Used

Without **LangChain**, each step would need its own custom code, and swapping any single part (like the AI model) would mean rewriting a lot of the system. **LangChain** was chosen because:

- It gives one common way to talk to different AI models, so the model can change without breaking the rest of the app.
- It already has ready-made tools for reading PDFs, splitting text, and creating **embeddings**, so this code did not need to be written from scratch.

- It is widely used for **RAG** projects, so it works well with both **ChromaDB** and **Google Gemini** out of the box.

4.2 How LangChain Is Implemented

All **LangChain** code lives in one backend file, **rag_service.py**. It is used at four points in the pipeline:

- Reading the PDF: **PyPDFLoader** reads the file page by page and keeps the page number attached to the text.
- Splitting the text: **RecursiveCharacterTextSplitter** cuts the text into 1000-character chunks with a 200-character overlap.
- Creating **embeddings**: **GoogleGenerativeAIEmbeddings** sends each chunk to **Gemini** and gets back a ready-to-store vector.
- Building the prompt: **LangChain** puts the **Top-5** matching chunks and the question together into one prompt and sends it to **Gemini** for the final answer.

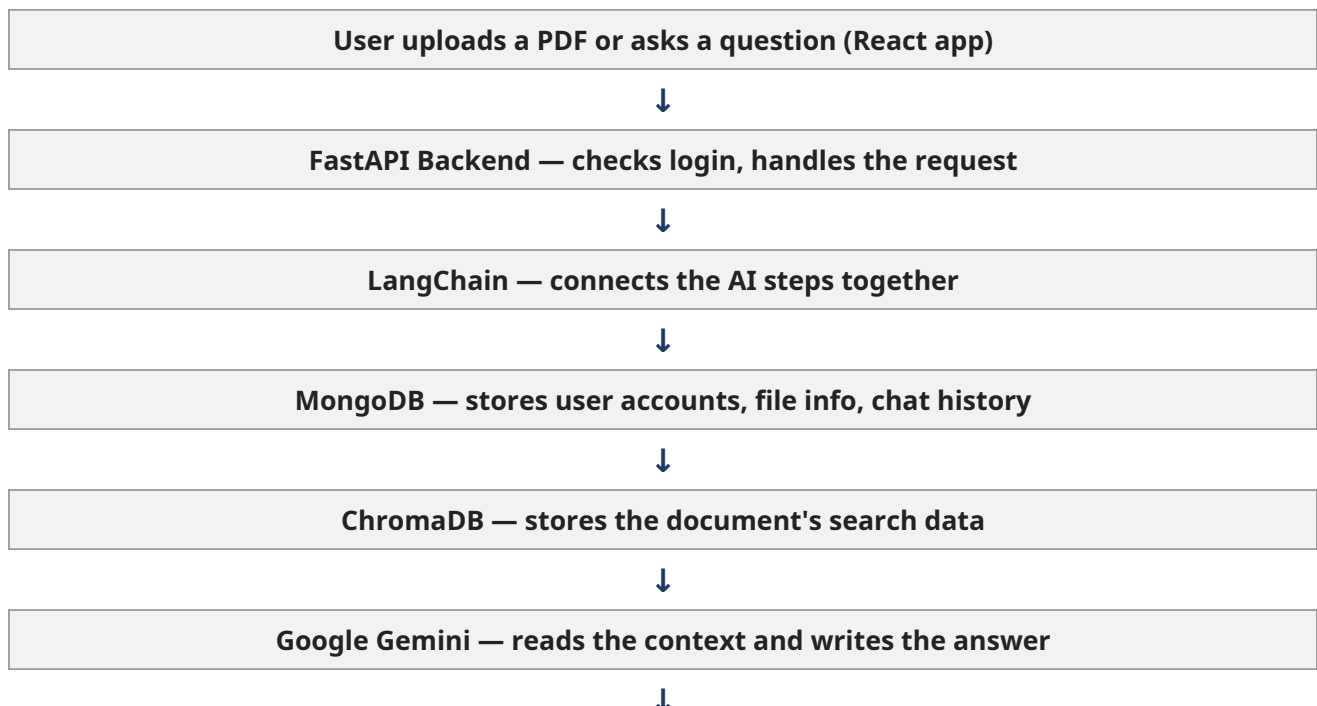
4.3 How LangChain Helped

LangChain saved a lot of time and reduced bugs, because:

- Page numbers stayed attached to the text automatically, so citations did not need extra tracking code.
- Switching AI models during testing meant changing a setting, not rewriting the whole pipeline.
- Connecting to **ChromaDB** took only a few lines of code instead of a custom database client.

5 System Architecture

The system has five simple parts that pass information to each other in order:



Answer + page number + source text sent back to the user

Part	What It Does
React (Frontend)	The app screen the user sees and types into.
FastAPI (Backend)	Handles requests, login, and routes data.
LangChain	Connects PDF reading, search, and the AI model into one flow.
MongoDB	Stores accounts, file details, and chat history.
ChromaDB	Stores the document's text as searchable data.
Google Gemini	Reads the context and writes the final answer.

6 Technology Stack

- Frontend: **React.js**, Vite, Tailwind CSS
- Backend: Python, **FastAPI**, Uvicorn
- AI Orchestration: **LangChain**
- AI Models: **Gemini 1.5 Flash** (for answers), **Gemini Embedding-001** (for search)
- Databases: **MongoDB** (accounts and chats), **ChromaDB** (document search data)

7 Document Processing Pipeline

When a PDF is uploaded, it goes through three simple steps before it can be searched:

- Ingestion: the file is saved on the server.
- Parsing: **LangChain's PyPDFLoader** reads the file page by page and keeps track of page numbers.
- Splitting: the text is cut into small pieces so the AI can search it easily.

8 Text Chunking Strategy

An **LLM** cannot read a huge file all at once, so the text is split into small pieces called chunks.

- Chunk size: 1000 characters.
- Chunk overlap: 200 characters.
- Why overlap? So a sentence split between two chunks is not lost. The end of one chunk repeats at the start of the next.

LangChain's RecursiveCharacterTextSplitter does this splitting automatically.

9 Embedding Generation

Computers do not understand words, only numbers. So each chunk of text is turned into a list of numbers called an **embedding**.

- Similar meanings get similar numbers. "The dog ran" and "The hound sprinted" end up close together.
- The **Gemini Embedding-001** model creates these **embeddings**.
- **LangChain** sends the text to **Gemini** and returns the **embedding** in a ready-to-store format.

10 Vector Database Architecture (ChromaDB)

ChromaDB is where all the **embeddings** are saved, so they can be searched later.

- Every PDF gets its own separate storage space (called a collection).
- This keeps documents fully separate, so one file's data can never mix with another's.
- Embeddings are saved to disk, not sent to an outside cloud service.

11 Semantic Search Mechanism

When a user asks a question, the system finds the most relevant chunks like this:

- The question is turned into an **embedding**, just like the PDF chunks were.
- **ChromaDB** compares the question to every chunk using **cosine similarity** (a way to measure closeness in meaning).
- The **Top-5** closest chunks are picked as the answer's source material.

This works better than keyword search because it understands meaning, not just exact words.

12 LLM-Based Question Answering

Once the **Top-5** chunks are found, they are given to **Gemini 1.5 Flash** to write the final answer.

- **LangChain** builds a prompt that includes the question and the 5 chunks.
- The prompt tells the model: use only this text, and say "I don't know" if the answer is not there.
- This stops the model from making things up.

13 Context Retrieval Workflow

Here is the full journey of one chat message, step by step:

1. User asks a question, e.g. "What is the conclusion of the report?"
2. The system turns the question into an **embedding**.
3. **ChromaDB** finds the 5 most relevant chunks.
4. The question and chunks are sent to **Gemini**.

- 5. **Gemini** writes the answer.
- 6. The backend pulls out the page numbers from those chunks.
- 7. The answer, page numbers, and source text are saved and sent back to the user.

14 Backend Implementation (FastAPI)

FastAPI runs the server. It is fast and handles many requests at once. The code is split into three simple layers:

- Routes: accept the incoming request from the app.
- Controllers: talk to **MongoDB** to save or fetch data.
- **rag_service.py**: one file that handles all the **LangChain**, **Gemini**, and **ChromaDB** work.

Keeping **LangChain** in one file makes the code easier to read, fix, and update later.

15 Frontend Implementation (React.js)

- A sidebar lets users manage more than one PDF at a time.
- Light and dark theme, saved using **React** Context.
- Chat replies show bold text, lists, and code blocks.
- A "Sources" panel lets users click and see the exact text the AI used.

16 Performance Evaluation

- Speed: a 50-page PDF is processed in seconds.
- Search speed: finding the right chunks takes milliseconds.
- Answer speed: **Gemini 1.5 Flash** is built for fast replies.
- Accuracy: the 1000/200 chunk setup with **Top-5** search gives accurate, low-**hallucination** answers for normal text PDFs.

17 Security and Privacy Considerations

- Login is protected using **JSON Web Tokens (JWT)**.
- Passwords are hashed with **bcrypt** before being saved.
- Every database check confirms the `user_id`, so no one can see another user's files.
- Embeddings stay on the local server disk, not on an outside vector cloud.

18 Use Cases

- Academic Research: search many papers at once and see what they all say about a topic.
- Legal & Compliance: summarize contracts, liabilities, or termination clauses.
- Human Resources: turn a resume into interview questions.

- Education: turn a textbook chapter into a quiz.

19 Advantages and Benefits

- Saves time: hours of reading become seconds of asking.
- Low **hallucination**: answers are grounded in the real document.
- Source proof: users can check the exact paragraph and page behind every answer.
- More than chat: it also summarizes, quizzes, and writes interview questions.

20 Limitations

- Tables, charts, and scanned PDFs are hard to read without **OCR**.
- Needs the internet and a valid **Gemini** API key to work.
- A very large file (1,000+ pages) may take a minute or two to process at first.

21 Future Scope

- Add **OCR** (using a tool like **Tesseract**) to read scanned PDFs.
- Let the AI read charts and images inside a PDF.
- Export quizzes and interview questions as a Word or PDF file.
- Let users chat with a live website by pasting a URL.

22 Conclusion

The Smart PDF Research Assistant turns slow document reading into fast, source-checked conversation. By combining **FastAPI**, **React**, **ChromaDB**, **LangChain**, and **Google Gemini**, it gives clear answers backed by real page numbers. It is simple to use and useful for research, education, legal work, and business analysis.

23 References

- [1] LangChain Documentation — <https://python.langchain.com>
- [2] Google Gemini API Documentation — Google AI for Developers
- [3] FastAPI Framework Documentation — <https://fastapi.tiangolo.com>
- [4] ChromaDB Vector Store Documentation — <https://www.trychroma.com>
- [5] React.js Library Documentation — <https://react.dev>